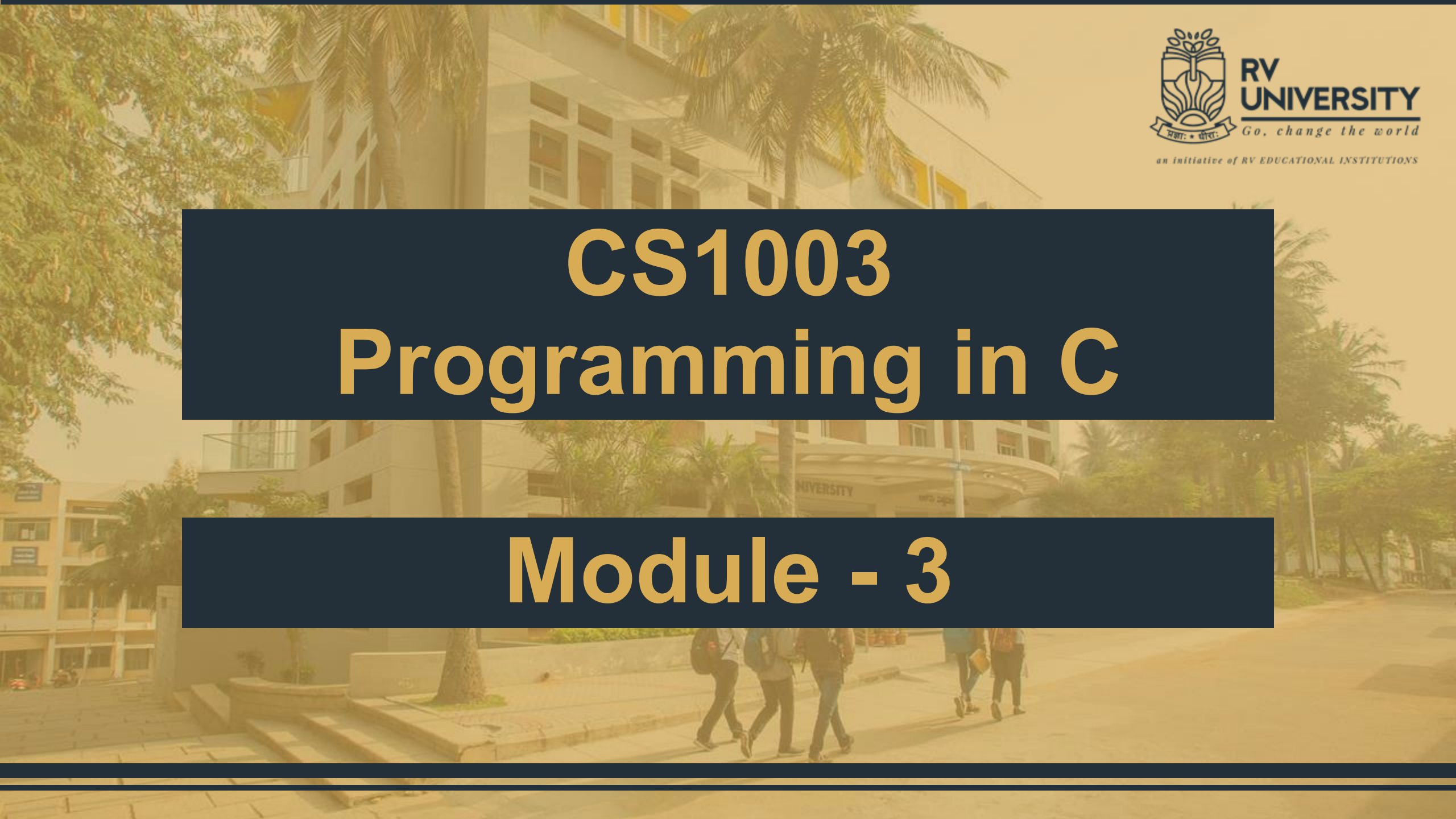

CS1003
Programming in C

Module - 3

Content of Module-3

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- **Chapter-1:**
- Decision Control in C in Conjunction with Operators in C.
- if Statement
- else-if Statement
- nested else -if Statement - Use case
- example for nested if-else, switch.

- **Chapter-2:**
- Loop statements: for, while, do-while
- break (from loop), continue
- exit - exit status to represent the exit code of the program
- Program clean ups done by exit function.

Content of Module-3

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- **Chapter-3:**
- Introduction to Functions – Declaration & Definition
- Function Prototype
- Calling a function
- Understanding the use of pointers
- Recursion

Decision Control in C in Conjunction with Operators in

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Decision control structures in C, such as if, else, switch, and looping constructs
- Allow you to control the flow of your program based on certain conditions.
- Operators play a crucial role in these decision-making processes
- if statement: Allows you to execute a block of code if a specified condition evaluates to true.
- Syntax:

```
if (condition)
{
 // Code to execute if condition is true
 }
```

Example:

```
int a = 5;
if (a > 0)
 { printf("a is positive\n");
 }
```

if-else statement

- The if-else statement extends the if statement to provide an alternative block of code if the condition is false.
- Syntax: if (condition) {

```
                    // Code to execute if condition is true
                      }
                else {
            // Code to execute if condition is false
                    }
```

Example:

```
int a = -3;
        if (a >= 0) {
                printf("a is non-negative\n");
                } else {
                        printf("a is negative\n");
                 }
```

else-if ladder

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Use multiple else if statements to check multiple conditions.
- Syntax:

```
if (condition1) {
        // Code for condition1
} else if (condition2) {
    // Code for condition2
} else {
        // Code if no conditions are true
}
```

Example:

```
int a = 75;
if (a >= 90) {
    printf("Grade: A\n");
} else if (a >= 80) {
    printf("Grade: B\n");
} else if (a >= 70) {
    printf("Grade: C\n");
} else {
    printf("Grade: F\n");
}
```

Switch statement

- The switch statement allows you to select one of many code blocks to execute based on the value of an expression.
- Syntax:

```
switch (expression) {
        case constant1:
         //Code to execute if expression == constant1
                break;
        case constant2:
       // Code to execute if expression == constant2
          break;
    default:
       // Code to execute if no case matches
}
```

Example:

```
int day = 3;
switch (day) {
    case 1:
        printf("Monday\n");
        break;
    case 2:
        printf("Tuesday\n");
        break;
    case 3:
        printf("Wednesday\n");
        break;
    default:
        printf("Weekend\n");
}
```

Nested if-else statement

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Used when you need to make multiple levels of decisions.
- Essentially, an if-else statement can contain another if-else statement inside it, allowing you to handle more complex conditions.
- Syntax:

```
if (condition1) {
  // Code to execute if condition1 is true
   if (condition2) {
      // Code to execute if condition2 is true
   } else {
      // Code to execute if condition2 is false
   }
} else {
   // Code to execute if condition1 is false
}
```

Example: Nested if-else statement

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

```
#include <stdio.h>
int main() {
    int score;
    printf("Enter the score (0-100): ");
    scanf("%d", &score);
    if (score >= 0 && score <= 100)
{ // Check if score is within valid range
        if (score >= 90) {
            printf("Grade: A\n");
        }
else if (score >= 80) {
            printf("Grade: B\n");
        } else if (score >= 70) {
            printf("Grade: C\n");
        } else if (score >= 60) {
            printf("Grade: D\n");
        } else {
            printf("Grade: F\n");
        }
    } else {
        printf("Invalid score\n"); // Handle invalid
score input
    }
    return 0;
}
```

Program to perform arithmetic operation using switch statement

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

```
#include <stdio.h>
int main() {
float num1, num2, result;
char operator;
printf("Enter two integer numbers: \n");
scanf("%f%f", &num1, &num2);
printf("Enter the operation (+, -, *, /): ");
scanf(" %c", &operator);
switch (operator) {
case '+':
result = num1 + num2;
printf("Sum: %f\n", result);

break;
 case '-‘: result = num1 - num2;
          printf("Difference: %f\n", result);
          break;
case '*‘:result = num1 * num2;
          printf("Product: %.2f\n", result);
          break;
case '/':// Check for division by zero
if (num2 != 0)
{
 result = num1 / num2;

 printf("Result: %.2f\n", result);

}
   else
   printf("Division by zero is not allowed.\n");

    break;
    default:
             printf("Invalid operator\n");
              break;
   }
```

Loop statements: for, while, do-while

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Loop statements are used to execute a block of code repeatedly based on a specified condition.
- The three primary types of loops are for, while, and do-while.
- Each type has its own syntax and use cases.

1. for() Loop: The for loop is commonly used when the number of iterations is known beforehand. It provides a concise way to initialize, test, and increment/decrement a loop control variable.
2. Syntax:

```
For (initialization; condition; increment/decrement) {
   // Code to execute in each iteration
   }
```

3. Example:

```
#include <stdio.h>
        int main() {
                for (int i = 1; i <= 5; i++) {
                                        printf("%d\n", i);    }
                return 0;}
```

While() Loop

- The while loop is used when you want to execute a block of code as long as a specified condition is true. The condition is checked before each iteration.
- Syntax: while (condition) {

```
// Code to execute as long as condition is true
        }
```

Example:

```
#include <stdio.h>
int main() {
int i = 1;  // Print numbers from 1 to 5
while (i <= 5) {
        printf("%d\n", i);
         i++; // Increment i
            }
return 0;
 }
```

do while() Loop

- The do-while loop is similar to the while loop but guarantees that the loop body is executed at least once because the condition is checked after the loop body.

➢Syntax:
```
do {
    // Code to execute
} while (condition);
```

Example:
```
#include <stdio.h>
int main() {
int i = 1;// Print numbers from 1 to 5
do {
    printf("%d\n", i);
    i++; // Increment i
} while (i <= 5);
return 0;
}
```

Difference between while and do while loops

Aspect	`while` Loop	`do-while` Loop
Condition Check	The condition is checked **before** executing the loop body.	The condition is checked **after** executing the loop body.
Guaranteed Execution	The loop body may not execute at all if the condition is false initially.	The loop body is guaranteed to execute at least once.
Typical Usage	Used when the number of iterations is not known in advance and you might want to skip the loop body if the condition is false initially.	Used when you need the loop body to execute at least once, regardless of the initial condition.
Syntax	`c while (condition) {  // Code to execute } `	`c do {  // Code to execute } while (condition); `
Example Scenario	Reading input until a valid input is provided (where the input might be invalid on the first check).	Displaying a menu where the menu should be shown at least once before checking if the user wants to exit.

Example:

```c
#include <stdio.h>

int main() {
    int i = 1;
    while (i <= 5) {
        printf("%d\n", i);
        i++;
    }
    return 0;
}
```

Example

```c
#include <stdio.h>

int main() {
    int i = 1;
    do {
        printf("%d\n", i);
        i++;
    } while (i <= 5);
    return 0;
}
```

Break statement

- break and continue are control flow statements used within loops (for, while, do-while) to alter their execution flow.
- The break statement is used to immediately **exit from the innermost loop** or switch statement in which it is placed.
- When break is executed, control is transferred to the statement immediately following the loop or switch block.
- Syntax: break;

Example:

```
#include <stdio.h>

int main() {
    for (int i = 1; i <= 10; i++) {
        if (i == 5) {
            break; // Exit the loop when i equals
5
        }
        printf("%d\n", i);
    }

    printf("Loop exited\n");
    return 0;
}
```

O/P :- 1 2 3 Loop exited

Continue statement

- The continue statement skips the remaining code inside the current iteration of the loop and proceeds to the next iteration.
- It is used to skip over certain iterations based on a condition.
- Syntax: continue:

Aspect	`break` Statement	`continue` Statement
Purpose	Exits from the innermost loop or `switch` statement.	Skips the remaining code in the current loop iteration and proceeds to the next iteration.
Scope	Can be used in `for`, `while`, `do-while` loops and `switch` statements.	Can be used in `for`, `while`, and `do-while` loops.
Effect	Ends the loop immediately and moves control to the statement following the loop.	Jumps to the next iteration of the loop, skipping the remaining code in the current iteration.

```
#include <stdio.h>

int main() {
    for (int i = 1; i <= 6; i++) {
        if (i % 2 == 0) {
            continue; // Skip the rest of the loop body
for even numbers
        }
        printf("%d\n", i);
    }

    return 0;
}
```

```
1 3 5
```

Exit() statement

- The exit function is used to terminate a program and return an exit status to the operating system.
- The exit function is part of the C Standard Library and is defined in the <stdlib.h> header file.
- Syntax: #include <stdlib.h>

  void exit(int status);
- status: An integer value that is returned to the operating system upon program termination.
- By convention:
  - 0 typically indicates successful completion.
  - Non-zero values indicate errors or abnormal termination.

Cont..

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

```
#include <stdio.h>

#include <stdlib.h>

int main() {

   printf("Program is starting...\n");

   // Simulate an error

   if (1) {

      printf("An error occurred.\n");

      exit(1); // Exit the program with status code 1

   }

   printf("This line will not be executed.\n");

      return 0;

}
```

Clean-up Actions Performed by exit

- When exit is called, several clean-up actions are performed automatically:
- **Flush and Close Output Buffers**:
  - Any buffered output (e.g., data held in stdout or stderr) is flushed, meaning it is written to the output destination.
  - Files opened with standard I/O functions (e.g., fopen) are closed.
- **Call Functions Registered with atexit**:
  - Functions registered with atexit are executed in reverse order of their registration.
  - These are typically used for clean-up tasks that need to be performed before the program exits.

```c
#include <stdio.h>
#include <stdlib.h>

void cleanup() {
    printf("Cleanup function called.\n");
}

int main() {
    atexit(cleanup); // Register cleanup function
    printf("Program is running...\n");
    exit(0);
}
```

Cont...

- **Release Resources:**
- Resources allocated dynamically using malloc, calloc, or realloc are not automatically released; you must manually free them before calling exit if you want to avoid memory leaks.
- However, the operating system reclaims all resources (e.g., memory, file descriptors) when the program terminates.
- **Terminate the Program:**
- The program is terminated, and control is returned to the operating system with the specified exit status.

Chapter-3 Introduction to Functions

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Introduction to Functions: Function Declaration and Definition
- Understanding how to declare a function, specify its return type, name, and parameter list, and define its implementation.
- Function Prototype - Declaration of the function's signature before its actual implementation, allowing for separate compilation and resolving any forward referencing issues.
- Calling a Function
- Return Values - Role of the return type in function declaration and definition.

Introduction to Functions: Function Declaration and Definition

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- A function is a block of code that performs a specific task
- Helps in structuring and organizing a program into manageable parts
- Advantages
- Breaking down complex problems into smaller, manageable pieces.
- Reusing code, reducing redundancy.
- Improving readability and maintainability of code

Function Syntax:

```
return_type function_name(parameter_list)
{
    // function body
    // statements
   return value;   // if return_type is not void
}
```

Function Declaration/function prototype

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- **Function Declaration** tells the compiler about the function's name, return type, and parameters before its actual definition.
- It allows the compiler to ensure that the function is used correctly throughout the code.

Function declaration:

return_type function_name(par_type1, par_name1, par_type2 par_name2, ...);

Example: int add(int a, int b);

Where: int is the return type.
add is the function name.
int a, int b are the parameters.

Function Definition

- **Function Definition** provides the actual implementation of the function. It contains the code that will be executed when the function is called.

Function definition Sysntax:

```
return_type function_name(par_type1 par_name1, par_type2 par_name2,
...)
{
   // function body
   // statements
   return value; // if return_type is not void
}
```

Example:

```
int add(int a, int b)
        {
          return a + b;
        }
```

Where:

- int is the return type
- add is the function name
- a and b are the parameters
- Function body calculates sum of a and b and returns the result

Function definition with Void

- If a function does not return a value, its return type is void.
- Example

```
void printMessage()
{
    printf("Hello, World!\n");
}
```

```
#include <stdio.h>

// Function Declaration
int multiply(int x, int y);  // Prototype

int main() {
    int result;
    result = multiply(3, 4);  // Function Call
    printf("The result is %d\n", result);
    return 0;
}

// Function Definition
int multiply(int x, int y) {
    return x * y;
}
```

Function call

- A function is called by using its name and passing the required arguments.
- The function call must match the function's declaration and definition in terms of the number and types of arguments.

```
#include <stdio.h>

// Function Declaration
int add(int a, int b);

int main() {
    int result;
    result = add(5, 3);  // Function Call
    printf("The result is %d\n", result);
    return 0;
}

// Function Definition
int add(int a, int b) {
    return a + b;
}
```

Return Types

- Functions can have various return types:

✓**int: Returns an integer.**

✓**float: Returns a floating-point number.**

✓**char: Returns a character.**

✓**void: Indicates that the function does not return any value.**

Parameter Lists

- **Parameters are optional. Functions can have no parameters.**
- **Parameters must be declared with a type and a name.**
- **If a function does not take any parameters, use empty parentheses.**

Summary

- Function Declaration: Tells the compiler about the function's name, return type, and parameters.
- Function Definition: Provides the implementation of the function, including the code to be executed.
- Function Call: Executes the function using its name and passing appropriate arguments.
- Return Types and Parameters: Define what type of value the function will return and the inputs it requires.

Function Prototype - Declaration of the function's signature before its actual implementation

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- A **function prototype** is a declaration of a function that specifies its name, return type, and parameters, but does not include the function's body.
- It provides a way for the compiler to know about the function's interface before its actual implementation is encountered in the code.
- This is particularly useful in ensuring type safety and proper function calls throughout the code.

- **Syntax of a Function Prototype**

```
return_type function_name(par_type1 par_name1, par_type2 par_name2, ...);
```

Where

- **return_type: Specifies the type of value that the function returns (e.g., int, float, void).**
- **function_name: The name of the function.**
- **parameter_type: The type of each parameter that the function accepts.**
- **parameter_name: The name of each parameter**

- **Function with Parameters and Return Type**
  - int add(int a, int b);
  - **The function add returns an int.**
  - **It takes two parameters, both of which are int.**

- **Function with No Parameters**
  - void printMessage(void);
  - The function printMessage returns void, meaning it does not return any value.
  - It does not take any parameters.

- **Function with a Different Return Type**
  - Float computeArea(float radius);
  - The function computeArea returns a float.
  - It takes one parameter of type float.

Purpose of Function Prototypes

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Forward Declaration:
  - Allows functions to be used before they are defined.
  - This is useful for organizing code and for defining functions that call each other.
- Type Checking:
  - Ensures that the function is called with the correct number and types of arguments.
  - If the function is called with mismatched arguments, the compiler will generate an error or warning.
- Code Readability:
  - Improves code readability and helps with understanding the functions available and their signatures without having to search through the entire codebase.

Demonstrating the use of function prototypes:

```
#include <stdio.h>
// Function Prototypes
int multiply(int x, int y);
void displayMessage(void);
    int main() {
    int result;
  // Function Calls
    result = multiply(4, 5);
    printf("The result is %d\n", result);
    displayMessage();
    return 0;
}
```

```
// Function Definitions
int multiply(int x, int y) {
    return x * y;
}
void displayMessage(void) {
    printf("Hello from the function!\n");
}
```

Allowing for separate compilation and resolving any forward referencing issues

- Helps in managing large codebases, enhancing modularity, and resolving issues related to function declarations and definitions.
- Separate Compilation
  - Process of dividing a program into multiple source files
  - Each of which can be compiled independently.
  - Helps in organizing code into manageable parts and facilitates code reuse.
  - The compiled object files are then linked together to create the final executable.
- How Separate Compilation Works:
- Source Files: The program is divided into multiple source files (.c files). Each file typically contains related functions and data.
- Header Files: Header files (.h files) contain function prototypes, macros, and declarations of global variables. They are included in source files that need to use these functions or variables.
- Compilation: Each source file is compiled into an object file (.o or .obj).
- Linking: The object files are linked together to produce the final executable.

Example

- Let's consider a simple program divided into two files: main.c and math_functions.c.

main.c

```
#include <stdio.h>
#include "math_functions.h"  //
Include the header file

int main() {
    int result = add(10, 5);
    printf("The result is %d\n",
result);
    return 0;
}
```

math.function.c

```
#include "math_functions.h"
// Include the header file

int add(int x, int y) {
    return x + y;
}
```

math.function.h

```
#ifndef MATH_FUNCTIONS_H
#define MATH_FUNCTIONS_H

int add(int x, int y);

#endif //
MATH_FUNCTIONS_H
```

- main.c includes math_functions.h to use the add function.
- math_functions.c contains the definition of the add function.
- math_functions.h declares the prototype of the add function.

Compilation Commands:

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- gcc -c main.c # Compile main.c into main.o
- gcc -c math_functions.c # Compile math_functions.c into math_functions.o
- gcc main.o math_functions.o -o myprogram # Link object files into an executable

Forward Referencing and Function Prototypes

- **Forward Referencing** occurs when a function is used before its definition is encountered in the code.
- This is common in larger programs where functions are often declared in header files and defined in separate source files.

- **Function Prototypes:**
  - A function prototype provides the compiler with information about the function's return type, name, and parameters before the actual function definition is encountered. This allows you to call functions before their definitions.
- Syntax: return_type function_name(parameter_type1 parameter_name1, ...);

main.c

```
#include <stdio.h>
#include "math_functions.h"

int main() {
int result = add(10, 5); // Function
call before the function definition
   printf("The result is %d\n", result);
   return 0;
}
```

math.function.h

```
#ifndef MATH_FUNCTIONS_H
#define MATH_FUNCTIONS_H

int add(int x, int y);  //
Function prototype

#endif //
MATH_FUNCTIONS_H
```

math.function.c

```
#include "math_functions.h"

int add(int x, int y) {  //
Function definition
   return x + y;
}
```

- main.c includes math_functions.h and calls add().
- math_functions.h declares the add function prototype.
- math_functions.c defines the add function.

Local, Global and Static Variables

- Variables are storage containers for data in a program which is used store, modify, and access data during program execution
- Local Variables are declared inside a function or a block. It is accessible only within the function/block they are declared in and it cannot be accessed outside of it.
- Local variables are created when the block or function is called and destroyed when the block or function exits.
- Example:

```
void localVariableFunction() {
   int x = 100; // Local variable
   printf("%d", x);
}
// 'x' cannot be accessed outside of 'localVariableFunction '.
```

Local, Global and Static Variables cont..

- A global variable is declared outside of all functions, usually at the top of a program.
- Global variables are accessible from any function within the same program.
- Global variables exist for the entire runtime of the program, from the time of declaration until the program terminates.
- Example:

```
int globalVariable = 5; // Global variable
void anyFunction() {
    globalVar = 10; // Access and modify the global variable
}
int main() {
    printf("%d", globalVar); // Access global variable
    return 0;
}
```

Local, Global and Static Variables cont..

- Static Variable can be declared locally or globally with the prefix to the variable declaration as “static”.
- A static local variable is declared within a function with a static keyword.
- Unlike regular local variables, static local variables retain their value across multiple function calls, meaning they are not destroyed when the function exits but persist for the program's lifetime
- Example:

```
void staiticLocalVariableFunction() {
    static int count = 0; // Static local variable
    count++;
    printf("%d", count);
}
```

Local, Global and Static Variables cont..

- If the static keyword is used as part of global variable, it is termed as static global variable.
- A static global variable exists for the entire runtime of the program.
- Example:

```
static int fileGlobal = 5; // Static global variable
void staticGlobalVariableFunction() {
    fileGlobal++;
}
```

Argument Passing

- Argument passing refers to how the data is being sent to functions for processing.
- There are two ways we can pass the arguments to functions
  - Pass by Value
  - Pass by Reference
- **Pass by Value**: In pass by value, a copy of the actual parameter (the variable's value) is passed to the function. This means that any changes made to the parameter inside the function do not affect the original variable.
- Example:

```
void passByValueFunction(int x) {
    x = 20;
}
int main() {
    int num = 10;
    passByValueFunction(num);
}
```

Argument Passing cont..

- **Pass by reference:** In this method, the address of an argument is passed to the function instead of the argument itself during the function call. Due to this, any changes made in the function will be reflected in the original arguments.
- We use the address operator(&) and indirection operator(*) to provide an argument to a function via reference.
- Example:

```
void passByReference(int *x) {
    *x = 20;
}
int main() {
    int num = 10;
 passByReference(&num);
}
```

Argument Passing cont..

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Difference between Pass by value and Pass by reference

Pass By Value	Pass By Reference
While calling a function, we pass the values of variables.	While calling a function, instead of passing the values of variables, we pass the address of variables
We cannot alter the values of actual variables through function calls.	We can alter the values of variables through function calls.
It is considered safer as original data is preserved	It is risky as it allows direct modification in original data

Pointers

- A pointer is a variable that stores the memory address of another variable. Instead of holding a data value directly, a pointer holds the location in memory where the data is stored.
- Example:
  - Declaring pointers
    - int *integerPointer; // Pointer to an integer
    - char *charPointer; // Pointer to a character
    - float *floatPointr; // Pointer to a float
  - Initializing Pointers
    - A pointer should be initialized with the address of a variable of the same type using the address-of operator (&).
      - int a = 10;
      - int *ptr = &a; // pointer now holds the address of var

Why Pointers

- Pointers allow functions to modify variables outside their own scope so inturn provides better efficiency
- Pointers save memory space. Execution time with pointers is faster because data are manipulated with the address, that is, direct access to memory location.
- It can be used to implement complex data structures such as linked lists, trees, and graphs.
- Pointers provide a way to iterate through arrays and manipulate strings efficiently.
- Pointers are essential for allocating and managing memory at runtime using functions like malloc, calloc, and free
- It can help in reducing the code and improving the performance.

Recursion

- **Definition**: Recursion is a programming technique where a function calls itself.

function calls.

- **Syntax**:

```
returnType functionName(parameters) {
        if (base_case_condition) {
            // Base case actions
            return base_case_value;
        } else {
            // Recursive case actions
            return functionName(modified_parameters);
        }
    }
```

Recursion

- **Base Case**: It is defined as a specific condition in the function, typically responsible for terminating the recursive function. When the base case is met, the recursive call stops and the function starts returning values. Therefore, it is always advisable to define base cases efficiently to avoid infinite recursions.
- **Recursive Case:** It is part of the function where it calls itself with a smaller or simpler argument.
- **Return Statement**: In a recursive function, it is possible to have multiple return statements that determine the value to be returned when the base case is reached.
- **Function Arguments** - In a recursive function, multiple arguments can be used, and they are modified in each recursive call to generate new outputs for subsequent calls

Pros and Cons of Recursion

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- **Pros**:
  - Some problems (like tree traversal, Fibonacci series, etc.) are easier to solve using recursion.
  - It requires a minimal number of programming statements.
  - Recursion breaks problems into smaller sub-problems.
  - Problems like factorial, Fibonacci, or those involving divide and conquer strategies (e.g., merge sort) can naturally be expressed recursively.
  - It is more useful in multiprogramming and multitasking environments.
- **Cons**:
  - Recursive calls consume more memory due to the function call stack, which could lead to stack overflow for large inputs.
  - Recursive logic can be harder to follow and debug than iterative solutions.

Example of Recursion

- **Factorial Problem:**

```
int factorial(int n) {
    if (n == 1) {  // Base case
        return 1;
    } else {  // Recursive case
        return n * factorial(n - 1);
    }
}
```

  - **Explanation:**
    - When n == 1, return 1(Base case)
    - Call factorial(n - 1) until n == 1(Recursive Case:)

Example of Recursion

- **GCD of Two Numbers using Recursion**

```
#include <stdio.h>
int hcf(int n1, int n2);
int main() {
 int n1, n2;
 printf("Enter two positive integers: ");
 scanf("%d %d", &n1, &n2);
 printf("G.C.D of %d and %d is %d.", n1, n2, hcf(n1, n2));
 return 0;
}
```

Output:

```
Enter two positive integers: 366
60
G.C.D of 366 and 60 is 6.
```

Example of Recursion

- **Fibonacci series using Recursion**

```
#include <stdio.h>
int fibonacci(int n) {
  if(n == 0)
    return 0;
  else if(n == 1)
    return 1;
  else
    return (fibonacci(n-1) + fibonacci(n-2));
}
```

```
int main() {
  int n;
        printf("Enter the number of terms\n");
  scanf("%d", &n);
  printf("Fibonacci Series: ");
  for (int i = 0; i < n; i++) {
    printf("%d ", fibonacci(i));
  }
  return 0;
}
```

OUTPUT: Enter the numbers of terms in the Fibonacci Series
10
The first 10 numbers in the Fibonacci Series are
0 1 1 2 3 5 8 13 21 34


RV
UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS
Thank You